Kosmocrates HYPHAE Master Specification v0.3

Implementation-Grade Hypercube-Guided Corpus Assimilation and Cube Swarm Coagulation for the Kosmocrates Workbench

Author: Sebastian Klemm

May 29, 2026

Document status. This document is an implementation-grade successor to the Kosmocrates HYPHAE Master Specification v0.2. It preserves the v0.2 doctrine of host-bound, evidence-first, non-copying topology assimilation, and extends it with a hypercube-native RepositoryCube model, SourceCube workers, CubeSwarm projection, Mandorla support intersection, Monolithic Coagulation, typed structural motifs, strict GateCascade decisions, and Kosmocrates/Workbench/Foundry integration.

Contents

Executive Summary v

Normative Language vi

Terminology vii

I Doctrine and Scope 1

1 Purpose and Scope 2
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 Non-goals . . . 2

2 Core Doctrine 4

3 Relationship to Kosmocrates Workbench 5
3.1 Layer Responsibilities . . . 5

II Formal Operational Model 6

4 HYPHAE Run Automaton 7
4.1 Canonical pipeline . . . 7
4.2 Run descriptor . . . 7

5 Formal Closure Rules 9

III Host Model and Deficiency 10

6 Host Binding 11
6.1 Host slot . . . 11

7 RepositoryCube and HostCube 12
7.1 RepositoryCube . . . 12

8 TopologicalVoidMap 14

9 DeficiencyVector 16
9.1 Purpose . . . 16

9.2 Priority model 17

IV Source Frontier and Acquisition 18

10 SourceFrontierGraph 19
10.1 Purpose 19
10.2 Ranking model 20

11 Source Acquisition 21
11.1 Acquisition pipeline 21

12 SourceEvidence 23

V Parsing and CodeHDAG 25

13 Parser Frontends 26
13.1 MVP frontends 26

14 CodeObservation 27

15 CodeHDAG Lowering 29

16 TopologySignature 31

VI RepositoryCube and CubeSwarm 32

17 CubeDimensionProfile 33
17.1 Default MVP profile 33

18 SourceCubeWorker 35

19 CubeSwarm 36

20 CubeMandorla 37

21 Monolithic Coagulation 38

22 CompositeSupportCube and HostTargetDelta 39

VII Motif Extraction 41

23 MotifCandidate 42

24 MotifStructuralCore 44

25 Motif Classes 45
25.1 TestDesignMotif 45
25.2 InterfaceContractMotif 45

25.3 LayeringMotif 45
25.4 SecurityBoundaryMotif 46

26 StructuralYield **47**

27 Norm and Anti-Pattern Drafts **49**

VIII Gates and Assimilation 51

28 GateCascade **52**
28.1 Gate ordering 52

29 AssimilationDecision **54**

30 Fail-Closed Doctrine **56**

IX Integration 57

31 Kosmocrates Evidence and Ledger **58**

32 Workbench ContextPack **59**

33 Pattern Memory and Norm Promotion **60**

34 Foundry Integration **61**

35 Retrieval and Negative Evidence **62**

X Implementation Architecture 63

36 Module Map **64**

37 API Contract **66**
37.1 Run request 66

38 CLI Contract **67**

39 Worker Orchestration **68**

40 MVP Implementation Cut **69**
40.1 MVP goal 69
40.2 MVP includes 69
40.3 MVP excludes 69

XI Metrics and Evaluation 71

41 Metric Layers **72**

42 CubeSwarm Metrics **73**

43 Void Reduction **74**

44 Success Criteria **75**

XII Roadmap and Definition of Done 76

45 Implementation Phases **77**

46 End-to-End MVP Scenario **78**
46.1 Scenario 78
46.2 Expected HYPHAE flow 78

47 System Definition of Done **79**

XIII Appendices 81

A Requirement Catalog **82**

B Gate Catalog **83**

C MVP Acceptance Tests **84**

D PlantUML Diagrams **85**
D.1 Run automaton 85
D.2 CubeSwarm 85

E Source Corpus Notes **87**

Closing Thesis **88**

Executive Summary

HYPHAE is the Kosmocrates Workbench function that converts external software-corpus fragmentation into governed structural advantage. It is not a scraper in the normative sense, not a code copier, not a trusted memory system, and not an autonomous modifier. Its standard path does not import external code into the host. Its standard path extracts topology.

A HYPHAE run binds a host repository, projects it into a HostCube, detects topological voids, derives a DeficiencyVector, builds a SourceFrontierGraph, acquires bounded external sources under capability locks, produces SourceEvidence, lowers eligible sources into CodeHDAGs, projects each source into a transient SourceCube, synchronizes SourceCubes as a CubeSwarm, computes a CubeMandorla, collapses resonant support through Monolithic Coagulation into a CompositeSupportCube, derives a HostTargetDelta, extracts typed motifs, gates all StructuralYields, and routes only evidence-bound outputs into Kosmocrates, Workbench, Agenda, Foundry, retrieval, or human review surfaces.

The v0.3 correction is the hypercube-native assimilation model. The v0.2 system described a topology-guided corpus assimilation layer centered around CodeHDAG and StructuralYield. The v0.3 system inserts a formal RepositoryCube layer between CodeHDAG and motif extraction. This makes the mechanism capable of processing many repositories in parallel as transient SourceCubes and deterministically coagulating their shared structural intersection relative to the HostCube.

The central doctrine is:

HYPHAE discovers reusable structure, not reusable authority.

External repositories may contribute evidence, topology, mesh signatures, fibers, phases, motifs, test designs, interface patterns, dependency patterns, anti-patterns, and norm candidates. They must not directly become trusted code, trusted memory, governance, tool authority, instruction authority, or authorized action.

Normative Language

The terms MUST, MUST NOT, SHOULD, and MAYare to be interpreted as requirement markers. A conformant implementation satisfies every MUST and MUST NOT statement. A SHOULD statement may be relaxed only with explicit evidence and an audit record. All non-normative explanatory prose is informative unless it includes one of these requirement markers.

Terminology

Term	Meaning
HYPHAE	Hypergraph-based Yielding Pattern Harvest and Assimilation Engine. In v0.3 it is the Workbench function for topology-first corpus assimilation.
HostCube	A RepositoryCube representing the bound host repository and its topological voids.
SourceCube	A transient RepositoryCube created from an acquired external source.
CubeSwarm	A run-local collection of SourceCubes projected against one HostCube under a shared CubeDimensionProfile.
CubeMandorla	The structural overlap/intersection region where multiple SourceCube projections provide compatible support for host voids.
Monolithic Coagulation	Deterministic collapse of CubeMandorla support into a CompositeSupportCube.
CompositeSupportCube	Evidence-bearing support object containing topology, mesh, fibers, motif regions, conflicts, scores, and provenance. It is not trusted memory.
HostTargetDelta	Structural target delta for filling HostCube voids. It is not a patch.
StructuralYield	Typed output derived from motif extraction and routed through GateCascade.
GateCascade	Ordered safety, provenance, topology, resonance, host-alignment, novelty, permitted-use, and promotion gates.
Foundry	Workbench validation path for executable effects: build, test, lint, typecheck, security scan, parse-back, and related evidence.

Part I

Doctrine and Scope

Chapter 1

Purpose and Scope

1.1 Purpose

The purpose of HYPHAE is to enable the Kosmocrates Workbench to learn structural advantage from external software corpora without copying external code or promoting external material into authority. The function binds a host repository, identifies topological gaps, searches external keyspaces for structurally relevant sources, extracts topology, projects sources into SourceCubes, coagulates multi-source support, and emits typed, evidence-bound candidates.

1.2 Scope

This specification covers:

- host binding and HostCube construction;
- TopologicalVoidMap and DeficiencyVector derivation;
- SourceFrontierGraph planning, ranking, and bounded expansion;
- capability-bound acquisition, sandboxing, source evidence, and negative evidence;
- parser frontends, CodeObservation, CodeHDAG, and TopologySignature;
- RepositoryCube, SourceCubeWorker, CubeSwarm, CubeMandorla, and Monolithic Coagulation;
- MotifCandidate, StructuralYield, AntiPatternEvidence, and NormCandidateDraft;
- GateCascade and AssimilationDecision;
- Kosmocrates, Workbench, Foundry, retrieval, agenda, and human review integration;
- metrics, telemetry, roadmap, MVP, and Definition of Done.

1.3 Non-goals

HYPHAE does not:

- train or replace language models;
- execute acquired repositories by default;

- autonomously mutate the host repository;
- import raw source code through the standard path;
- implement a second trusted memory substrate;
- promote patterns or norms by discovery alone;
- bypass Kosmocrates governance, capability locks, taint, license, provenance, or Foundry gates.

Chapter 2

Core Doctrine

DOCTRINE-001
HYPHAE MUST discover reusable structure, not reusable authority.

DOCTRINE-002
External material MAY contribute evidence, topology, motifs, tests, interfaces, anti-patterns, and norm candidates. It MUST NOT directly become trusted code, trusted memory, governance, tool authority, instruction authority, or authorized action.

DOCTRINE-003
Consensus, source count, resonance, CubeMandorla membership, or CubeSwarm convergence MUST NOT bypass authority, taint, license, provenance, governance, capability, or Foundry gates.

DOCTRINE-004
The standard HYPHAE path is non-copying. Raw external source code MUST NOT enter host patches, default Workbench prompts, trusted memory, governance objects, or tool authorization through this path.

Chapter 3

Relationship to Kosmocrates Workbench

3.1 Layer Responsibilities

Layer	Responsibility
Kosmocrates Core	Epistemic substrate: Infinity Ledger, Crystals, HDAG, QTIC, governance, evidence, agenda, replay, retrieval.
Kosmocrates Workbench	Production substrate: TaskSpec, WorkspaceIndex, ContextPack, agent runtime, ActionCandidate, isolated mutation, Foundry, operator UX.
HYPHAE	Workbench function for external structural assimilation: frontier, acquisition, CodeHDAG, RepositoryCubes, CubeSwarm, motifs, evidence-bound candidates.
Foundry	Executable validation through build, tests, lint, typecheck, security scans, parse-back, and repair-loop evidence.
Human operator	Goal authorization, high-risk review, release posture, governance-sensitive approvals.

`REL-001`
HYPHAE MUST emit outputs through Kosmocrates evidence and Workbench integration surfaces. It MUST NOT directly mutate host files, trusted memory, governance, git state, tool authority, or capability locks.

`REL-002`
HYPHAE MAY propose PatternCandidates and NormCandidateDrafts. Final promotion belongs to Kosmocrates governance and memory semantics.

Part II

Formal Operational Model

Chapter 4

HYPHAE Run Automaton

4.1 Canonical pipeline

```
RunDescriptor
-> HostBinding
-> HostCube
-> TopologicalVoidMap
-> DeficiencyVector
-> SourceIntent
-> SourceFrontierGraph
-> AcquisitionEligibility
-> SourceAcquisitionCapability
-> SourceEvidence
-> CodeObservation
-> CodeHDAG
-> SourceCubeWorker
-> SourceCube
-> CubeSwarm
-> CubeMandorla
-> MonolithicCoagulation
-> CompositeSupportCube
-> HostTargetDelta
-> MotifCandidate
-> StructuralYield
-> GateCascade
-> AssimilationDecision
-> IntegrationOutput
-> Metrics / LearningTrace
```

4.2 Run descriptor

```
pub struct HyphaeRunDescriptor {
    pub run_id: Digest,
    pub host_project_id: Digest,
    pub task_spec_id: Option<Digest>,
```

```
    pub goal: HyphaeGoal,
    pub mode: HyphaeRunMode,
    pub profile_id: Digest,
    pub policy_profile_id: Digest,
    pub genesis_crystal_id: Digest,
    pub budget: HyphaeBudget,
    pub created_at: Timestamp,
}
```

```
pub enum HyphaeGoal {
    FillMissingTests,
    DiscoverArchitectureMotifs,
    RepairCompileFailurePattern,
    ImproveLayerSeparation,
    DiscoverInterfaceContracts,
    FindSecurityBoundaryPatterns,
    FindPluginArchitecturePatterns,
    GeneralDeficiencyAssimilation,
}
```

RUN-001
Every HYPHAE run MUST have a RunDescriptor bound to HostProject, policy profile, Genesis Crystal, budget, and goal.

RUN-002
A HYPHAE run without valid Genesis binding MUST operate only in report-only/read-only mode.

Chapter 5

Formal Closure Rules

CLOSURE-001 Operational Closure
Every HYPHAE transition MUST have typed inputs, typed outputs, and explicit failure states.

CLOSURE-002 Evidence Closure
Every output MUST be traceable to run descriptor, host binding, source evidence or host evidence, policy, gate trace, and evidence bundle.

CLOSURE-003 Authority Closure
External material MAY become evidence, but MUST NOT become instruction, trusted memory, governance, tool authority, secret access, or direct patch.

CLOSURE-004 Cube Closure
SourceCubes MAY be synchronized and coagulated only as topology, mesh, fibers, phases, motifs, conflicts, scores, and evidence. They MUST NOT be merged as source code.

CLOSURE-005 Learning Closure
Positive and negative outcomes SHOULD be retrievable for future DeficiencyVector, SourceFrontier, acquisition ranking, MotifScoring, and GateCascade decisions.

Part III

Host Model and Deficiency

Chapter 6

Host Binding

6.1 Host slot

```
pub struct HostSlot {
    pub host_project: HostProject,
    pub workspace_digest: Digest,
    pub vcs_head: Option<String>,
    pub language_inventory: BTreeMap<Language, usize>,
    pub dependency_graph_digest: Digest,
    pub test_inventory_digest: Digest,
    pub symbol_index_digest: Digest,
}
```

```
pub struct HostStructuralState {
    pub host_id: Digest,
    pub workspace_digest: Digest,
    pub code_hdag_digest: Digest,
    pub language_inventory: BTreeMap<Language, usize>,
    pub dependency_graph_digest: Digest,
    pub test_inventory_digest: Digest,
    pub api_surface_digest: Option<Digest>,
    pub build_graph_digest: Option<Digest>,
    pub known_defects: Vec<DefectRef>,
    pub unresolved_agenda_items: Vec<AgendaItemId>,
}
```

`HOST-001`
Every default HYPHAE run MUST bind to a host repository or explicit target project.

`HOST-002`
Generic scraping without host binding and deficiency root MUST NOT occur in Workbench mode.

Chapter 7

RepositoryCube and HostCube

7.1 RepositoryCube

```
pub struct RepositoryCube {
    pub id: Digest,
    pub cube_kind: RepositoryCubeKind,

    pub profile_id: Digest,
    pub state_vector: StateVector5D,

    pub embedded_hdag_id: Digest,
    pub mesh: CubeMesh,
    pub layers: Vec<CubeLayer>,
    pub temporal_fibers: Vec<TemporalFiber>,
    pub semantic_fibers: Vec<SemanticFiber>,

    pub void_map: TopologicalVoidMap,
    pub topology_signature: TopologySignature,

    pub source_evidence_id: Option<Digest>,
    pub host_project_id: Option<Digest>,

    pub evidence_refs: Vec<EvidenceRef>,
    pub taint: TaintLabel,
    pub license_status: LicenseUseStatus,
}
```

```
pub enum RepositoryCubeKind {
    HostCube,
    SourceCube,
    CompositeSupportCube,
    MotifCube,
}
```

A RepositoryCube is a layered 5D phase container with an embedded CodeHDAG mesh. The 5D coordinates are not physical dimensions. They are profile-bound cybernetic and

topological measurement axes.

`CUBE-001`
Every HostRepository MUST be representable as a HostCube before CubeSwarm assimilation.

`CUBE-002`
Every acquired repository that passes acquisition gates SHOULD be lowered into a transient SourceCube.

`CUBE-003`
HostCube and SourceCube comparison is valid only when both are projected into the same CubeDimensionProfile or through an explicit ProfileTranslationMap.

Chapter 8

TopologicalVoidMap

```
pub struct TopologicalVoidMap {
    pub id: Digest,
    pub cube_id: Digest,
    pub voids: Vec<TopologicalVoid>,
    pub aggregate_void_score: Q16,
}
```

```
pub struct TopologicalVoid {
    pub id: Digest,
    pub region: CubeRegionRef,
    pub void_kind: VoidKind,

    pub missing_layers: Vec<CubeLayerKind>,
    pub missing_fibers: Vec<ExpectedFiber>,
    pub weak_edges: Vec<CodeEdgeKind>,
    pub sparse_roles: Vec<RoleKind>,

    pub severity: Q16,
    pub confidence: Q16,
    pub assimilation_priority: Q16,
}
```

```
pub enum VoidKind {
    MissingLayer,
    SparseMesh,
    WeakSemanticCoupling,
    BrokenTemporalSequence,
    MissingTestFiber,
    MissingInterfaceFiber,
    MissingSecurityFiber,
    PhaseDisalignment,
    HighEntropyRegion,
}
```

A topological void is the host-side assimilation target. It is the region where the HostCube's mesh is sparse, misphased, insufficiently fibered, under-tested, weakly coupled, or otherwise

structurally incomplete.

`VOID-001`
Every Workbench-usable StructuralYield MUST reference at least one HostVoid or DeficiencyItem.

`VOID-002`
HostVoid detection SHOULD record severity, confidence, missing layers, missing fibers, weak edges, sparse roles, and assimilation priority.

Chapter 9

DeficiencyVector

9.1 Purpose

The DeficiencyVector is the operative search form derived from HostCube voids and Workbench goals. It decides what the SourceFrontier is allowed to search for.

```
pub struct DeficiencyVector {
    pub id: Digest,
    pub host_id: Digest,
    pub run_id: Digest,
    pub goal: HyphaeGoal,
    pub items: Vec<DeficiencyItem>,
    pub aggregate_scores: DeficiencyAggregateScores,
    pub source_intents: Vec<SourceIntent>,
    pub evidence_refs: Vec<EvidenceRef>,
    pub created_at: Timestamp,
}
```

```
pub struct DeficiencyItem {
    pub id: Digest,
    pub kind: DeficiencyKind,
    pub scope: DeficiencyScope,

    pub affected_nodes: Vec<HDAGNodeRef>,
    pub affected_edges: Vec<HDAGEdgeRef>,
    pub affected_files: Vec<PathRef>,

    pub evidence: Vec<EvidenceRef>,

    pub severity: Q16,
    pub confidence: Q16,
    pub centrality: Q16,
    pub fixability: Q16,
    pub searchability: Q16,
    pub memory_coverage: Q16,

    pub priority: Q16,
```

```
    pub search_intent: SearchIntent,
}
```

9.2 Priority model

```
priority =
    severity
  * confidence
  * centrality
  * fixability
  * searchability
  * goal_weight
  * policy_weight
  * (1.0 - memory_coverage)
```

DEF-001
A HYPHAE run MUST NOT launch external acquisition before a DeficiencyVector has been computed, loaded, or explicitly approved.

DEF-002
An LLM MAY propose a deficiency label, but it MUST NOT be the sole evidence for a DeficiencyItem that launches acquisition.

DEF-003
Every SourceIntent MUST derive from at least one DeficiencyItem.

Part IV

Source Frontier and Acquisition

Chapter 10

SourceFrontierGraph

10.1 Purpose

The SourceFrontierGraph is a replayable, bounded traversal graph over permitted external and internal keyspaces.

```
pub struct SourceFrontierGraph {
    pub id: Digest,
    pub run_id: Digest,
    pub host_id: Digest,
    pub deficiency_vector_id: Digest,

    pub root_intents: Vec<SourceIntentRef>,
    pub nodes: Vec<FrontierNode>,
    pub edges: Vec<FrontierEdge>,

    pub policy: SourcePolicy,
    pub budget: AcquisitionBudget,
    pub ranking_profile: FrontierRankingProfile,

    pub expansion_state: FrontierExpansionState,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum FrontierNodeKind {
    QuerySeed,
    SearchQuery,
    SearchResult,

    Repository,
    Package,
    Crate,
    NpmPackage,
    PyPiPackage,

    DocumentationPage,
    ExampleDirectory,
```

```
    TestDirectory,
    IssueThread,
    PullRequest,
    ReleaseArtifact,

    LocalCorpusItem,
    ApprovedMirrorItem,

    PriorPositiveEvidence,
    PriorNegativeEvidence,

    RejectedSource,
    QuarantinedSource,
}
```

FRONTIER-001
Every FrontierGraph MUST derive from at least one SourceIntent.

FRONTIER-002
Every SearchQuery MUST record its derivation path.

FRONTIER-003
HYPHAE SHOULD rank and filter source candidates using metadata before downloading full source content.

FRONTIER-004
A FrontierNode MUST NOT transition to AcquisitionScheduled without a valid SourceAcquisitionCapability.

10.2 Ranking model

```
pub struct FrontierNodeRanking {
    pub deficiency_alignment: Q16,
    pub topology_likelihood: Q16,
    pub source_quality: Q16,
    pub source_diversity: Q16,
    pub novelty: Q16,
    pub license_safety: Q16,
    pub acquisition_cost: Q16,
    pub taint_risk: Q16,
    pub prior_negative_penalty: Q16,
    pub total_score: Q16,
}
```

Chapter 11

Source Acquisition

11.1 Acquisition pipeline

```
A0 Select AcquisitionEligible frontier nodes
A1 Check SourceAcquisitionCapability
A2 Create isolated acquisition sandbox
A3 Fetch metadata or source snapshot
A4 Normalize source identity
A5 Compute content digest
A6 Detect license
A7 Scan secrets
A8 Scan prompt-injection or instruction-like content
A9 Classify taint
A10 Emit SourceEvidence
A11 Queue allowed material for parsing and lowering
A12 Persist failures as negative evidence
```

```
pub struct SourceAcquisitionCapability {
    pub id: Digest,
    pub run_id: Digest,
    pub frontier_graph_id: Digest,
    pub genesis_crystal_id: Digest,
    pub policy_profile_id: Digest,

    pub allowed_keyspaces: Vec<Keyspace>,
    pub allowed_domains: Vec<String>,
    pub forbidden_domains: Vec<String>,

    pub max_sources: usize,
    pub max_bytes_total: u64,
    pub max_bytes_per_source: u64,
    pub max_depth: usize,

    pub requires_license_detection: bool,
    pub requires_secret_scan: bool,
    pub requires_injection_scan: bool,
    pub requires_digest: bool,
```

```
    pub expires_at: Option<Timestamp>,
}
```

ACQ-001
Every acquisition MUST reference a valid SourceAcquisitionCapability.

ACQ-002
Every acquired source MUST be stored outside the host repository in an isolated acquisition sandbox.

ACQ-003
Acquired sources MUST NOT be executed by default. Build scripts, install scripts, package hooks, tests, examples, and generated commands from acquired sources MUST NOT run during acquisition.

Chapter 12

SourceEvidence

```
pub struct SourceEvidence {
    pub id: Digest,
    pub run_id: Digest,
    pub frontier_node_id: Digest,
    pub acquisition_request_id: Digest,

    pub snapshot: SourceSnapshot,
    pub file_manifest_id: Digest,

    pub license_report_id: Digest,
    pub secret_scan_report_id: Digest,
    pub injection_scan_report_id: Digest,
    pub taint_report_id: Digest,

    pub acquisition_trace_id: Digest,
    pub evidence_bundle_id: Digest,

    pub parse_eligibility: ParseEligibility,
    pub permitted_use: SourcePermittedUse,
}
```

```
pub enum ParseEligibility {
    ParseAllowed,
    ParseAllowedRedacted,
    MetadataOnly,
    DocumentationSummaryOnly,
    QuarantineNoParse,
}
```

SOURCE-001
Every acquired source MUST bind URI, canonical URI, timestamp, commit or version if available, file manifest digest, and content digest.

SOURCE-002
Every acquired source MUST receive a LicenseReport before parsing-derived yield may

become a candidate.

SOURCE-003
Sources with suspected secrets MUST be quarantined or redacted before further processing.

SOURCE-004
Instruction-like external text MUST be labeled and MUST NOT become instruction authority.

Part V

Parsing and CodeHDAG

Chapter 13

Parser Frontends

```rust
pub trait ParserFrontend {
    fn language(&self) -> Language;
    fn version(&self) -> ParserVersion;
    fn supported_extensions(&self) -> Vec<String>;

    fn parse_file(
        &self,
        file: &SourceFileRef,
        content: &[u8],
        policy: &ParsePolicy,
    ) -> Result<ParsedFile, ParseError>;

    fn extract_observations(
        &self,
        parsed: &ParsedFile,
        context: &ParseContext,
    ) -> Result<Vec<CodeObservation>, ObservationError>;
}
```

PARSE-001
Canonical CodeObservation and CodeHDAG construction MUST NOT require an LLM.

PARSE-002
Parser frontends MUST preserve provenance, taint, and license state.

13.1 MVP frontends

The MVP SHOULD implement Rust, TOML/Cargo manifest parsing, JSON/YAML config parsing, and Markdown documentation classification as external text evidence. TypeScript, Python, and SQL MAY follow in Phase B.

Chapter 14

CodeObservation

```
pub struct CodeObservation {
    pub id: Digest,
    pub source_evidence_id: Digest,
    pub file_ref: SourceFileRef,

    pub language: Language,
    pub observation_kind: ObservationKind,

    pub symbol: Option<SymbolRef>,
    pub span: Option<ByteSpan>,
    pub line_span: Option<LineSpan>,

    pub raw_name: Option<String>,
    pub normalized_name: Option<String>,
    pub signature: Option<SignatureShape>,

    pub attributes: BTreeMap<String, ObservationValue>,
    pub confidence: Q16,
}
```

```
pub enum ObservationKind {
    File,
    Module,
    Package,
    DependencyManifest,
    BuildTarget,

    FunctionDecl,
    MethodDecl,
    TypeDecl,
    StructDecl,
    EnumDecl,
    TraitDecl,
    InterfaceDecl,
    ImplBlock,
```

```
    ImportDecl,
    ExportDecl,
    CallExpr,
    ConstructorCall,
    TypeReference,
    TraitImplementation,
    InterfaceImplementation,

    RouteDecl,
    HandlerDecl,
    MiddlewareDecl,

    TestDecl,
    TestFixture,
    Assertion,
    MockOrStub,

    TableDecl,
    MigrationDecl,
    QueryDecl,
    RepositoryDecl,

    ConfigKey,
    SecretAdjacentConfig,
    ToolInvocationDecl,

    ErrorType,
    ErrorHandlingBranch,
    ValidationBranch,

    DocumentationBlock,
    ExampleBlock,
}
```

Chapter 15

CodeHDAG Lowering

```
pub struct CodeHDAG {
    pub id: Digest,
    pub source_evidence_id: Digest,

    pub nodes: Vec<CodeNode>,
    pub edges: Vec<CodeEdge>,

    pub graph_digest: Digest,
    pub topology_signature: TopologySignature,

    pub provenance: EvidenceBundleId,
    pub taint: TaintLabel,
    pub license_status: LicenseUseStatus,

    pub parse_completeness: Q16,
    pub confidence: Q16,
}
```

```
pub enum CodeEdgeKind {
    Contains,
    Imports,
    Exports,
    Calls,

    Implements,
    Instantiates,
    Extends,
    DependsOn,

    Provides,
    Consumes,
    Returns,
    Accepts,

    Tests,
    Fixtures,
```

```
    Asserts,

    RoutesTo,
    Handles,
    Validates,
    Serializes,
    Deserializes,

    Queries,
    Migrates,
    Configures,

    Publishes,
    Subscribes,
    Adapts,
    Wraps,

    FailsWith,
    RecoversWith,
}
```

HDAG-001
Canonical topology MUST use typed CodeNodes and typed CodeEdges.

HDAG-002
Name/path-only inference MUST be marked as heuristic and MUST NOT alone justify high-trust assimilation.

HDAG-003
CodeHDAG construction SHOULD produce a stable graph digest under fixed parser versions, source snapshot, parse policy, and normalization profile.

Chapter 16

TopologySignature

```
pub struct TopologySignature {
    pub graph_digest: Digest,

    pub size: GraphSizeSignature,
    pub role: RoleSignature,
    pub layer: LayerSignature,
    pub dependency: DependencySignature,
    pub test: TestSignature,
    pub api: ApiSignature,
    pub data: DataSignature,
    pub error: ErrorSignature,
    pub spectral: SpectralSignature,

    pub quality: TopologyQuality,
}
```

SIG-001
A final topology signature MUST be multi-axis. A single Jaccard score MUST NOT be used as the final basis for trusted assimilation.

SIG-002
Where numerical metrics influence gates, deterministic fixed-point or otherwise replay-stable arithmetic SHOULD be used.

Part VI

RepositoryCube and CubeSwarm

Chapter 17

CubeDimensionProfile

```
pub struct CubeDimensionProfile {
    pub id: Digest,
    pub name: String,
    pub axes: [AxisBinding; 5],
    pub normalization: NormalizationProfile,
    pub distance_metric: DistanceMetric,
    pub resonance_metric: ResonanceMetric,
}
```

```
pub struct AxisBinding {
    pub symbol: String,
    pub semantic_role: AxisSemanticRole,
    pub measurement_operator: MeasurementOperatorId,
    pub range: NumericRange,
}
```

17.1 Default MVP profile

Axis	RepositoryAssimilation5D binding
D1 / psi	Structural potential: unresolved improvement mass in host-relative topology.
D2 / rho	Mesh density: relation, edge, role, and fiber density.
D3 / omega	Phase rhythm: lifecycle sequencing, test/build/repair cadence, temporal ordering.
D4 / chi	Connectivity: topological coupling, reachability, layer connectivity, boundary cohesion.
D5 / eta	Causal clarity and uncertainty residue: how clearly causes, effects, tests, errors, and interfaces are ordered.

PROFILE-001
A HYPHAE cube MUST use exactly five active model coordinates per profile.

`PROFILE-002`
The semantic meaning of all five axes MUST be declared by CubeDimensionProfile.

`PROFILE-003`
The 5D vector alone is not sufficient for assimilation. It is a routing and resonance surface over the underlying CodeHDAG, evidence, roles, edges, fibers, phases, and deficiency regions.

Chapter 18

SourceCubeWorker

```
pub struct SourceCubeWorker {
    pub id: Digest,
    pub run_id: Digest,
    pub source_evidence_id: Digest,
    pub frontier_node_id: Digest,

    pub assigned_phase: PhaseSlot,
    pub profile_id: Digest,
    pub bounds: WorkerBounds,

    pub status: WorkerStatus,
}
```

```
pub enum WorkerStatus {
    Pending,
    Parsing,
    LoweringToHDAG,
    ProjectingToCube,
    Completed,
    Failed,
    Quarantined,
}
```

WORKER-001
SourceCubeWorkers MAY execute in parallel.

WORKER-002
Parallel execution MUST NOT change final CubeSwarm, CompositeSupportCube, MotifCandidate, StructuralYield, or AssimilationDecision output.

WORKER-003
Worker merge order MUST be canonicalized by deterministic digest ordering, not by wall-clock completion order.

Chapter 19

CubeSwarm

```
pub struct CubeSwarm {
    pub id: Digest,
    pub run_id: Digest,
    pub host_cube_id: Digest,
    pub profile_id: Digest,

    pub source_cube_ids: Vec<Digest>,
    pub worker_traces: Vec<WorkerTraceRef>,

    pub phase_schedule: PhaseSchedule,
    pub projection_policy: ProjectionPolicy,
    pub convergence_policy: ConvergencePolicy,
}
```

```
pub struct CubeProjection {
    pub id: Digest,
    pub source_cube_id: Digest,
    pub host_cube_id: Digest,
    pub profile_id: Digest,

    pub projected_regions: Vec<ProjectedRegion>,
    pub resonance_edges: Vec<CubeResonanceEdge>,
    pub conflict_edges: Vec<CubeConflictEdge>,

    pub projection_quality: ProjectionQuality,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

SWARM-001
Every SourceCube in a CubeSwarm MUST declare the same active CubeDimensionProfile or an explicit translation into that profile.

SWARM-002
SourceCube conflicts MUST be preserved. Conflicting regions MUST NOT be silently discarded.

Chapter 20

CubeMandorla

```
pub struct CubeMandorla {
    pub id: Digest,
    pub host_cube_id: Digest,
    pub profile_id: Digest,

    pub support_regions: Vec<SupportRegion>,
    pub rejected_regions: Vec<RejectedRegion>,
    pub conflict_regions: Vec<ConflictRegion>,

    pub total_support: Q16,
    pub total_conflict: Q16,
    pub source_diversity: Q16,
}
```

```
pub struct SupportRegion {
    pub id: Digest,
    pub host_void_ref: TopologicalVoidRef,
    pub source_region_refs: Vec<CubeRegionRef>,

    pub support_count: usize,
    pub source_diversity: Q16,
    pub mesh_density_gain: Q16,
    pub fiber_consensus: Q16,
    pub phase_coherence: Q16,
    pub conflict_score: Q16,

    pub accepted: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

Support is not majority vote. Support is a product of source count, diversity, resonance, host fit, low conflict, evidence quality, and gate eligibility.

Chapter 21

Monolithic Coagulation

```
pub struct MonolithicCoagulation {
    pub id: Digest,
    pub run_id: Digest,
    pub host_cube_id: Digest,
    pub cube_swarm_id: Digest,
    pub mandorla_id: Digest,

    pub convergence_score: Q16,
    pub threshold: Q16,
    pub decision: CoagulationDecision,

    pub composite_support_cube_id: Option<Digest>,
    pub host_target_delta_id: Option<Digest>,

    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum CoagulationDecision {
    EmitCompositeSupportCube,
    EmitPartialSupport,
    RequireMoreSources,
    EmitConflictEvidenceOnly,
    RejectLowResonance,
    AbortPolicyViolation,
}
```

COAG-001
MonolithicCoagulation MUST NOT merge raw code. It may merge only topology, roles, edges, fibers, phases, void coverage, signatures, scores, conflicts, and evidence.

COAG-002
If convergence threshold is not met, HYPHAE MUST emit partial evidence, conflict evidence, negative evidence, or request more sources instead of forcing a CompositeSupportCube.

Chapter 22

CompositeSupportCube and HostTargetDelta

```
pub struct CompositeSupportCube {
    pub id: Digest,
    pub host_cube_id: Digest,
    pub profile_id: Digest,

    pub support_mesh: CubeMesh,
    pub support_regions: Vec<SupportRegion>,
    pub motif_regions: Vec<MotifRegion>,
    pub conflict_regions: Vec<ConflictRegion>,

    pub density_gain: Q16,
    pub coherence_gain: Q16,
    pub void_coverage: Q16,
    pub source_diversity: Q16,

    pub evidence_refs: Vec<EvidenceRef>,
    pub taint_summary: TaintSummary,
    pub license_summary: LicenseSummary,
}
```

```
pub struct HostTargetDelta {
    pub id: Digest,
    pub host_cube_id: Digest,
    pub composite_support_cube_id: Digest,

    pub filled_voids: Vec<TopologicalVoidRef>,
    pub proposed_layers: Vec<LayerDelta>,
    pub proposed_fibers: Vec<FiberDelta>,
    pub proposed_edges: Vec<EdgeDelta>,
    pub proposed_motifs: Vec<MotifRef>,

    pub expected_effect: ExpectedCubeEffect,
    pub risk: DeltaRiskProfile,
    pub evidence_refs: Vec<EvidenceRef>,
```

```
}
```

DELTA-001
HostTargetDelta MUST reference the HostVoid regions it claims to fill.

DELTA-002
CompositeSupportCube is not trusted memory. It is an evidence-bearing support object.

Part VII

Motif Extraction

Chapter 23

MotifCandidate

```
pub struct MotifCandidate {
    pub id: Digest,
    pub run_id: Digest,

    pub host_cube_id: Digest,
    pub host_void_refs: Vec<TopologicalVoidRef>,
    pub composite_support_cube_id: Digest,

    pub motif_kind: MotifKind,
    pub structural_core: MotifStructuralCore,

    pub support: MotifSupport,
    pub alignment: HostAlignment,
    pub novelty: NoveltyScore,
    pub conflict: ConflictScore,
    pub risk: MotifRiskProfile,

    pub evidence_refs: Vec<EvidenceRef>,
    pub taint_summary: TaintSummary,
    pub license_summary: LicenseSummary,
}
```

```
pub enum MotifKind {
    LayeringMotif,
    InterfaceContractMotif,
    TestDesignMotif,
    ErrorRepairMotif,
    DependencyIntegrationMotif,
    SecurityBoundaryMotif,
    APISurfaceMotif,
    DataModelMotif,
    BuildPipelineMotif,
    PluginArchitectureMotif,
    ConfigBoundaryMotif,
    ObservabilityMotif,
    AntiPatternMotif,
```

```
    UnknownStructuralMotif,
}
```

MOTIF-001
Every MotifCandidate MUST derive from SourceCube, CubeMandorla, CompositeSupportCube, HostTargetDelta, or prior Kosmocrates evidence.

MOTIF-002
Every Workbench-usable MotifCandidate MUST reference at least one HostVoid or DeficiencyItem.

Chapter 24

MotifStructuralCore

```
pub struct MotifStructuralCore {
    pub role_pattern: RolePattern,
    pub edge_pattern: EdgePattern,
    pub fiber_pattern: FiberPattern,
    pub layer_pattern: LayerPattern,
    pub phase_pattern: Option<PhasePattern>,
    pub hdag_fragment: CodeHDAGFragment,
    pub cube_region_refs: Vec<CubeRegionRef>,
}
```

```
pub struct MotifSupport {
    pub support_count: usize,
    pub source_count: usize,
    pub source_diversity: Q16,

    pub mandorla_membership: Q16,
    pub mesh_density_gain: Q16,
    pub fiber_consensus: Q16,
    pub phase_coherence: Q16,

    pub average_source_quality: Q16,
    pub evidence_quality: Q16,
}
```

Chapter 25

Motif Classes

25.1 TestDesignMotif

```
pub struct TestDesignMotif {
    pub target_role: RoleKind,
    pub required_test_roles: Vec<TestRoleKind>,
    pub fixture_pattern: Option<FixturePattern>,
    pub assertion_pattern: AssertionPattern,
    pub negative_case_matrix: Vec<NegativeCase>,
    pub regression_binding: Option<FoundryFailureClass>,
}
```

25.2 InterfaceContractMotif

```
pub struct InterfaceContractMotif {
    pub boundary_role: RoleKind,
    pub consumer_roles: Vec<RoleKind>,
    pub provider_roles: Vec<RoleKind>,
    pub required_methods_shape: Vec<SignatureShape>,
    pub error_contract: Option<ErrorContractShape>,
    pub substitution_rule: InterfaceSubstitutionRule,
}
```

25.3 LayeringMotif

```
pub struct LayeringMotif {
    pub layer_sequence: Vec<CubeLayerKind>,
    pub allowed_cross_edges: Vec<LayerEdgeRule>,
    pub forbidden_cross_edges: Vec<LayerEdgeRule>,
    pub adapter_rules: Vec<AdapterRule>,
}
```

25.4 SecurityBoundaryMotif

```
pub struct SecurityBoundaryMotif {
    pub boundary_kind: SecurityBoundaryKind,
    pub protected_flow: Vec<RoleKind>,
    pub required_checks: Vec<SecurityCheckKind>,
    pub taint_flow_rule: TaintFlowRule,
    pub forbidden_shortcuts: Vec<EdgePattern>,
}
```

Chapter 26

StructuralYield

```
pub struct StructuralYield {
    pub id: Digest,
    pub run_id: Digest,

    pub source: StructuralYieldSource,
    pub yield_kind: YieldKind,
    pub payload: YieldPayload,

    pub host_alignment: HostAlignment,
    pub support: MotifSupport,
    pub novelty: NoveltyScore,
    pub conflict: ConflictScore,
    pub risk: MotifRiskProfile,

    pub evidence_refs: Vec<EvidenceRef>,
    pub taint_summary: TaintSummary,
    pub license_summary: LicenseSummary,

    pub permitted_next_steps: Vec<PermittedNextStep>,
}
```

```
pub enum YieldPayload {
    CodeHDAGFragment(CodeHDAGFragment),
    LayeringMotif(LayeringMotif),
    InterfaceContractMotif(InterfaceContractMotif),
    TestDesignMotif(TestDesignMotif),
    ErrorRepairMotif(ErrorRepairMotif),
    DependencyIntegrationMotif(DependencyIntegrationMotif),
    SecurityBoundaryMotif(SecurityBoundaryMotif),
    PluginArchitectureMotif(PluginArchitectureMotif),
    AntiPatternDraft(AntiPatternDraft),
    NormCandidateDraft(NormCandidateDraft),
    ContextHint(ContextHint),
}
```

YIELD-001
StructuralYield MUST be typed. Opaque textual yield is invalid.

YIELD-002
Raw source code MUST NOT be emitted as StructuralYield.

YIELD-003
Every StructuralYield MUST carry taint and license summaries.

Chapter 27

Norm and Anti-Pattern Drafts

```
pub struct NormCandidateDraft {
    pub id: Digest,
    pub name: String,
    pub norm_kind: NormKind,

    pub predicate_shape: NormPredicateShape,
    pub scope: NormScope,
    pub trigger_patterns: Vec<TriggerPattern>,

    pub derived_from_motifs: Vec<MotifCandidateRef>,
    pub mandorla_ref: Option<CubeMandorlaRef>,
    pub support: MotifSupport,

    pub conflicts: Vec<NormConflictRef>,
    pub required_validation: Vec<ValidationStep>,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub struct AntiPatternDraft {
    pub id: Digest,
    pub anti_pattern_kind: AntiPatternKind,
    pub observed_shape: MotifStructuralCore,
    pub risk_explanation: String,
    pub affected_host_voids: Vec<TopologicalVoidRef>,
    pub source_support: MotifSupport,
    pub avoidance_rule: Option<NormPredicateShape>,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

NORM-001
NormCandidateDraft MUST require stronger support than PatternCandidate, including multi-source support or CubeMandorla membership.

ANTI-001
AntiPatternEvidence MUST be preserved when it explains host risk or future search

avoidance.

Part VIII

Gates and Assimilation

Chapter 28

GateCascade

```
pub struct GateCascade {
    pub id: Digest,
    pub run_id: Digest,
    pub structural_yield_id: Digest,

    pub gate_profile_id: Digest,
    pub gate_results: Vec<GateResult>,

    pub final_decision: AssimilationDecision,
    pub final_permitted_use: Option<PermittedUse>,

    pub evidence_bundle_id: Digest,
}
```

```
pub struct GateResult {
    pub gate_id: GateId,
    pub gate_kind: GateKind,
    pub status: GateStatus,

    pub score: Option<Q16>,
    pub threshold: Option<Q16>,

    pub blocking: bool,
    pub reason: String,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

28.1 Gate ordering

```
G0  Run / Genesis Gate
G1  Host Binding Gate
G2  Source Evidence Gate
G3  Provenance Completeness Gate
G4  License Gate
G5  Secret Gate
```

```
G6  Injection Gate
G7  Taint Gate
G8  Non-Copying / Originality Gate
G9  Topology Validity Gate
G10 Cube Resonance Gate
G11 Host Alignment Gate
G12 Conflict Gate
G13 Novelty / Duplication Gate
G14 Workbench Permitted-Use Gate
G15 Memory Promotion Eligibility Gate
G16 Foundry Requirement Gate
G17 Human Review Gate
G18 Final Assimilation Decision
```

GATE-001
Every StructuralYield MUST pass GateCascade before use.

GATE-002
Hard safety gates MUST execute before usefulness gates.

GATE-003
Every GateResult MUST emit evidence.

Chapter 29

AssimilationDecision

```
pub enum AssimilationDecision {
    Quarantine,
    RejectWithNegativeEvidence,
    EvidenceOnly,
    AntiPatternEvidence,
    ContextHint,
    TestMotifCandidate,
    InterfaceMotifCandidate,
    PatternCandidate,
    NormCandidateDraft,
    AgendaItem,
    HumanReviewRequired,
}
```

```
pub struct CandidateEvidenceRecord {
    pub id: Digest,
    pub run_id: Digest,
    pub structural_yield_id: Digest,

    pub decision: AssimilationDecision,
    pub permitted_use: Option<PermittedUse>,

    pub gate_cascade_id: Digest,
    pub evidence_bundle_id: Digest,

    pub kosmocrates_target: Option<KosmocratesTarget>,
    pub workbench_target: Option<WorkbenchTarget>,
    pub agenda_target: Option<AgendaTarget>,
}
```

ASSIM-001
AssimilationDecision MUST be one of the declared decision classes.

ASSIM-002
Workbench-usable output MUST declare permitted use.

`ASSIM-003`
Final trusted memory promotion is outside HYPHAE and belongs to Kosmocrates.

Chapter 30

Fail-Closed Doctrine

FAIL-001
Unknown safety state MUST NOT promote.

FAIL-002
Missing provenance MUST NOT promote.

FAIL-003
InjectionCandidate content MUST NOT become instruction.

FAIL-004
SecretAdjacent content MUST NOT enter Workbench ContextPack except as redacted EvidenceOnly.

FAIL-005
Unvalidated executable influence MUST NOT become high-trust pattern.

FAIL-006
Consensus, support count, resonance, or Mandorla membership MUST NOT bypass governance, authority, license, taint, capability, or Foundry gates.

Part IX

Integration

Chapter 31

Kosmocrates Evidence and Ledger

```
pub struct HyphaeEvidenceBundle {
    pub id: Digest,
    pub run_id: Digest,

    pub run_descriptor_digest: Digest,
    pub source_frontier_ledger_id: Option<Digest>,
    pub acquisition_trace_ids: Vec<Digest>,
    pub source_evidence_ids: Vec<Digest>,
    pub code_hdag_ids: Vec<Digest>,
    pub source_cube_ids: Vec<Digest>,

    pub composite_support_cube_id: Option<Digest>,
    pub structural_yield_id: Option<Digest>,
    pub gate_cascade_id: Digest,
    pub final_decision: AssimilationDecision,

    pub taint_summary: TaintSummary,
    pub license_summary: LicenseSummary,
    pub created_at: Timestamp,
}
```

INT-EVIDENCE-001
Every AssimilationDecision MUST produce or reference an EvidenceBundle.

INT-LEDGER-001
Every HYPHAE run MUST emit a replayable ledger trace.

Chapter 32

Workbench ContextPack

```
pub struct HyphaeContextPackContribution {
    pub id: Digest,
    pub task_spec_id: Digest,
    pub host_project_id: Digest,
    pub items: Vec<HyphaeContextItem>,
    pub evidence_bundle_id: Digest,
}
```

```
pub struct HyphaeContextItem {
    pub id: Digest,
    pub source_yield_id: Digest,
    pub kind: HyphaeContextItemKind,
    pub permitted_use: PermittedUse,
    pub payload_ref: Digest,
    pub summary: String,
    pub taint: TaintLabel,
    pub license_status: LicenseUseStatus,
}
```

INT-CTX-001
HYPHAE ContextPack contributions MUST be bounded, typed, attributed, and permitted-use labeled.

INT-CTX-002
Raw external source code MUST NOT enter default Workbench ContextPacks.

Chapter 33

Pattern Memory and Norm Promotion

```
pub struct PatternMemoryProposal {
    pub id: Digest,
    pub source_yield_id: Digest,
    pub proposal_kind: PatternMemoryProposalKind,
    pub motif_core: MotifStructuralCore,
    pub support: MotifSupport,
    pub host_alignment: HostAlignment,
    pub validation_requirements: Vec<ValidationStep>,
    pub evidence_bundle_id: Digest,
}
```

INT-NORM-001
HYPHAE MAY propose PatternCandidates and NormCandidateDrafts.

INT-NORM-002
HYPHAE MUST NOT promote them to trusted norms.

Chapter 34

Foundry Integration

```
pub struct FoundryValidationRequest {
    pub id: Digest,
    pub source_yield_id: Digest,
    pub host_project_id: Digest,
    pub validation_kind: FoundryValidationKind,
    pub proposed_effect: ExpectedCubeEffect,
    pub required_checks: Vec<FoundryCheck>,
    pub evidence_bundle_id: Digest,
}
```

```
pub enum FoundryCheck {
    Build,
    UnitTests,
    IntegrationTests,
    Lint,
    TypeCheck,
    SecurityScan,
    SnapshotDiff,
    ParseBackVerification,
}
```

INT-FOUNDRY-001
Any HYPHAE output that may influence executable code MUST declare a Foundry validation path before it can become high-trust pattern memory.

Chapter 35

Retrieval and Negative Evidence

```
pub struct HyphaeRetrievalIndexRecord {
    pub id: Digest,
    pub object_digest: Digest,
    pub object_kind: HyphaeRetrievalObjectKind,

    pub host_project_id: Option<Digest>,
    pub deficiency_kinds: Vec<DeficiencyKind>,
    pub motif_kinds: Vec<MotifKind>,
    pub cube_profile_id: Option<Digest>,

    pub success_score: Option<Q16>,
    pub negative_reason: Option<NegativeEvidenceReason>,
    pub evidence_bundle_id: Digest,
}
```

INT-RETRIEVAL-001
HYPHAE MUST make successful and failed assimilation traces retrievable for future DeficiencyVector, SourceFrontier, acquisition, MotifScoring, and GateCascade decisions.

Part X

Implementation Architecture

Chapter 36

Module Map

```
kosmo-hyphae/
  run.rs
  host.rs
  deficiency.rs
  void_map.rs
  frontier.rs
  acquisition.rs
  sandbox.rs
  source_evidence.rs
  parser_frontend.rs
  parsing.rs
  observation.rs
  hdag.rs
  topology_signature.rs
  cube.rs
  cube_profile.rs
  cube_mesh.rs
  cube_projection.rs
  cube_swarm.rs
  coagulation.rs
  motif.rs
  structural_yield.rs
  gates.rs
  assimilation.rs
  integration.rs
  context_pack.rs
  foundry.rs
  retrieval.rs
  ledger.rs
  metrics.rs
  policy.rs
  errors.rs
```

IMPL-001
HYPHAE MUST be implemented as a deterministic, evidence-emitting, Workbench-

integrated subsystem.

`IMPL-002`
HYPHAE MUST NOT depend on an LLM provider for canonical parsing, CodeHDAG lowering, Cube projection, gate decisions, or evidence generation.

`IMPL-003`
HYPHAE MAY use an internal working store for active runs. This working store MUST NOT be treated as trusted memory.

Chapter 37

API Contract

```
POST   /hyphae/runs
GET    /hyphae/runs/{run_id}
GET    /hyphae/runs/{run_id}/events
GET    /hyphae/runs/{run_id}/evidence
GET    /hyphae/runs/{run_id}/frontier
GET    /hyphae/runs/{run_id}/cubes
GET    /hyphae/runs/{run_id}/yields
POST   /hyphae/runs/{run_id}/cancel
POST   /hyphae/runs/{run_id}/approve-review
```

37.1 Run request

```
{
  "host_project_id": "sha256:...",
  "task_spec_id": "sha256:...",
  "goal": "FillMissingTests",
  "mode": "OnDemand",
  "profile_id": "RepositoryAssimilation5D",
  "policy_profile_id": "default-local-safe",
  "budget": {
    "max_sources": 20,
    "max_bytes_total": 500000000,
    "max_workers": 8,
    "max_frontier_depth": 2,
    "max_cycles": 1
  }
}
```

Chapter 38

CLI Contract

```
kosmo hyphae run --host . --goal fill-missing-tests
kosmo hyphae run --host . --goal discover-architecture-motifs --max-sources 50
kosmo hyphae status <run-id>
kosmo hyphae evidence <run-id>
kosmo hyphae frontier <run-id>
kosmo hyphae cubes <run-id>
kosmo hyphae yields <run-id>
kosmo hyphae export-report <run-id>
```

Chapter 39

Worker Orchestration

```
pub enum HyphaeWorkerKind {
    FrontierSearchWorker,
    AcquisitionWorker,
    ParseWorker,
    SourceCubeWorker,
    ProjectionWorker,
    MotifWorker,
    GateWorker,
}
```

WORKER-ORCH-001
Worker execution MAY be concurrent.

WORKER-ORCH-002
All semantic selection and merge steps MUST use deterministic ordering.

WORKER-ORCH-003
Wall-clock completion order MUST NOT influence final SourceCube ordering, support scoring, motif selection, or assimilation decision.

Chapter 40

MVP Implementation Cut

40.1 MVP goal

HYPHAE MVP v0.3 is a safe, Rust-first, on-demand Workbench function that derives host-relevant test and architecture motifs from bounded external or local repositories through non-copying topology extraction, SourceCube projection, deterministic CubeSwarm coagulation, GateCascade, and evidence-backed Workbench integration.

40.2 MVP includes

- HostBinding;
- DeficiencyVector;
- SourceFrontierGraph;
- bounded Acquisition;
- SourceEvidence;
- Rust CodeHDAG;
- HostCube and SourceCube;
- deterministic CubeSwarm merge;
- CompositeSupportCube or NoConvergenceEvidence;
- TestDesignMotif, LayeringMotif, InterfaceContractMotif;
- GateCascade;
- CandidateEvidenceRecord;
- Workbench ContextPack contribution;
- EvidenceBundle and Ledger trace;
- no raw external code in default prompt/context.

40.3 MVP excludes

- unbounded crawling;
- autonomous host mutation;

- direct trusted memory promotion;
- raw code import through standard path;
- executing acquired repositories;
- background daemon as default;
- external source as instruction authority.

Part XI

Metrics and Evaluation

Chapter 41

Metric Layers

HYPHAE success is measured through structural utility under evidence, safety, replay, and governance constraints. Source count alone is not success.

1. Run Health
2. Frontier Efficiency
3. Acquisition Safety
4. Parsing / CodeHDAG Quality
5. CubeSwarm Quality
6. Motif / Yield Quality
7. Gate / Assimilation Quality
8. Workbench / Foundry Impact
9. Learning / Autonomy Delta

```
pub struct HyphaeMetrics {
    pub run: RunHealthMetrics,
    pub frontier: FrontierMetrics,
    pub acquisition: AcquisitionMetrics,
    pub parsing: ParsingMetrics,
    pub cube_swarm: CubeSwarmMetrics,
    pub motifs: MotifMetrics,
    pub gates: GateMetrics,
    pub integration: IntegrationMetrics,
    pub impact: ImpactMetrics,
    pub learning: LearningMetrics,
}
```

MET-001
HYPHAE success MUST NOT be measured by source count alone.

MET-002
Every run SHOULD report frontier, acquisition, parsing, cube, motif, gate, integration, impact, and learning metrics.

Chapter 42

CubeSwarm Metrics

```
pub struct CubeSwarmMetrics {
    pub host_void_count_initial: usize,
    pub source_cubes_built: usize,
    pub source_cube_failure_rate: Q16,

    pub cube_projection_success_rate: Q16,
    pub avg_projection_quality: Q16,
    pub avg_source_resonance: Q16,
    pub avg_phase_coherence: Q16,
    pub avg_fiber_consensus: Q16,

    pub mandorla_support_regions: usize,
    pub conflict_regions: usize,

    pub composite_support_emitted: bool,
    pub composite_convergence_score: Q16,
    pub composite_density_gain: Q16,
    pub composite_coherence_gain: Q16,
    pub composite_void_coverage: Q16,

    pub deterministic_swarm_digest_mismatch_count: usize,
}
```

MET-CUBE-001
CubeSwarm metrics MUST include resonance, phase coherence, fiber consensus, source diversity, conflict score, and void coverage.

Chapter 43

Void Reduction

```
pub struct VoidReductionMetrics {
    pub host_void_count_before: usize,
    pub host_void_count_after_predicted: usize,
    pub host_void_count_after_foundry: Option<usize>,

    pub predicted_void_reduction: Q16,
    pub foundry_validated_void_reduction: Option<Q16>,

    pub voids_filled_by_kind: BTreeMap<VoidKind, usize>,
    pub voids_unresolved_by_kind: BTreeMap<VoidKind, usize>,
}
```

Predicted void reduction is not the same as Foundry-validated void reduction. The first is HYPHAE's structural forecast. The second is confirmed by actual Workbench/Foundry use.

MET-VOID-001
Executable impact MUST be separated into predicted impact and Foundry-validated impact.

Chapter 44

Success Criteria

A HYPHAE run is successful if:

1. evidence completeness meets threshold;
2. no safety hard gate is bypassed;
3. at least one host-aligned StructuralYield is emitted or useful negative evidence is persisted;
4. Workbench-usable outputs are typed and permitted-use scoped;
5. executable influence is Foundry-bound;
6. repeated similar tasks show reduced search waste or improved memory hit rate.

Part XII

Roadmap and Definition of Done

Chapter 45

Implementation Phases

Phase	Goal
0	Specification Freeze.
A	Core Types and Run Orchestrator.
B	Frontier and Memory Precheck.
C	Safe Acquisition and SourceEvidence.
D	Rust-first Parsing and CodeHDAG.
E	HostCube, SourceCube and VoidMap.
F	CubeSwarm and Monolithic Coagulation.
G	Motif Extraction and StructuralYield.
H	GateCascade and Assimilation Decisions.
I	Kosmocrates / Workbench / Foundry Integration.
J	Gateway, CLI, Telemetry and Reports.

Chapter 46

End-to-End MVP Scenario

46.1 Scenario

A Rust web API host repository lacks integration tests for routes and input validation.

46.2 Expected HYPHAE flow

1. HostBinding scans workspace.
2. HostCube detects MissingTestFiber and WeakSecurityBoundary.
3. DeficiencyVector creates SourceIntents for Rust API test structures.
4. Frontier finds bounded Rust API repositories or examples.
5. Acquisition fetches sources into sandbox.
6. CodeHDAG extracts routes, handlers, tests, fixtures, assertions.
7. SourceCubeWorkers project each repository into RepositoryAssimilation5D.
8. CubeSwarm aligns SourceCubes against HostVoids.
9. CompositeSupportCube identifies recurring route-test fixture motif.
10. MotifExtractor emits TestDesignMotif.
11. GateCascade classifies it as TestMotifCandidate.
12. Workbench receives TestDesignHint.
13. FoundryValidationRequest is created for generated tests.
14. EvidenceBundle and Ledger trace close the run.

Chapter 47

System Definition of Done

DOD-001
A complete HYPHAE run MUST be replayable from RunDescriptor, HostBinding, SourceFrontierGraph, SourceEvidence, CodeHDAGs, CubeSwarm artifacts, StructuralYields, GateCascade, and Integration outputs.

DOD-002
Every acquired source MUST have digest, license, secret, injection, taint, and provenance records.

DOD-003
Every Workbench-usable output MUST be typed, host-bound, permitted-use scoped, and evidence-backed.

DOD-004
Raw external source code MUST NOT enter default Workbench prompts, host patches, trusted memory, governance, or tool authorization.

DOD-005
Parallel SourceCubeWorkers MUST NOT change semantic output.

DOD-006
CompositeSupportCube and HostTargetDelta MUST contain only topology, mesh, fiber, phase, motif, conflict, score, and evidence structures.

DOD-007
Every StructuralYield MUST pass GateCascade.

DOD-008
Consensus, source count, or resonance MUST NOT bypass safety, authority, license, taint, governance, or Foundry gates.

DOD-009
Executable influence MUST be Foundry-bound.

`DOD-010`
Positive and negative outcomes SHOULD be retrievable for future HYPHAE runs.

`DOD-011`
At least one end-to-end MVP scenario MUST prove the host improvement path: MissingIntegrationTest to Frontier to SourceCubes to CompositeSupportCube to TestDesignMotif to GateCascade to ContextPack to FoundryValidationRequest.

Part XIII

Appendices

Appendix A

Requirement Catalog

ID	Requirement summary
DOCTRINE-001	HYPHAE discovers reusable structure, not reusable authority.
DOCTRINE-002	External material may inform but not become authority.
DOCTRINE-003	Resonance and consensus cannot bypass gates.
RUN-001	Every run has a typed RunDescriptor.
HOST-001	Every default run is host-bound.
DEF-001	No external acquisition before DeficiencyVector.
FRONTIER-001	Every FrontierGraph derives from SourceIntent.
ACQ-001	Every acquisition references capability.
SOURCE-001	Every acquired source has identity and digest.
PARSE-001	Canonical extraction does not require LLM.
HDAG-001	Canonical topology uses typed nodes and edges.
PROFILE-001	Cubes use exactly five active model coordinates per profile.
WORKER-002	Parallel execution cannot change final semantic output.
COAG-001	Coagulation cannot merge raw code.
YIELD-001	StructuralYield is typed.
GATE-001	Every StructuralYield passes GateCascade.
FAIL-006	Consensus cannot bypass governance gates.
INT-CTX-002	Raw external code cannot enter default ContextPack.
IMPL-002	HYPHAE does not depend on LLM provider for canonical mechanics.
MET-001	Source count alone is not success.
DOD-011	End-to-end MissingIntegrationTest scenario must pass.

Appendix B

Gate Catalog

Gate	Purpose
G0	Validate run, Genesis, policy, and descriptor.
G1	Ensure host binding and deficiency relation.
G2	Ensure source evidence exists.
G3	Ensure provenance chain completeness.
G4	Evaluate license status and permitted use.
G5	Block or redact secret findings.
G6	Label and block injection-like instruction promotion.
G7	Propagate and enforce taint.
G8	Enforce non-copying and originality.
G9	Validate topology, CodeHDAG, cube, and motif core.
G10	Evaluate cube resonance, phase coherence, fiber consensus, source diversity, and conflict.
G11	Evaluate HostVoid / Deficiency alignment.
G12	Preserve and respond to conflicts.
G13	Detect duplicate, known rejected, or refining motifs.
G14	Determine Workbench permitted use.
G15	Determine memory-promotion eligibility only.
G16	Declare Foundry validation requirement.
G17	Require human review when policy cannot safely classify.
G18	Emit final AssimilationDecision.

Appendix C

MVP Acceptance Tests

1. Run without Genesis stays report-only.
2. HYPHAE run without HostBinding fails.
3. External acquisition without capability is blocked.
4. Sandbox cannot write to host repository.
5. Acquired source receives digest, license, secret, injection, and taint reports.
6. Raw external source code never appears in default ContextPack.
7. Rust parser emits deterministic CodeHDAG digest for fixed source snapshot.
8. Two parallel worker schedules produce identical CompositeSupportCube digest.
9. Low-license-safety yield becomes EvidenceOnly or Reject.
10. Secret finding causes quarantine.
11. Injection-like external text cannot become instruction.
12. SourceCube conflict is preserved, not silently discarded.
13. MotifCandidate without HostVoid cannot become Workbench-usable.
14. NormCandidateDraft cannot become trusted Norm directly.
15. Foundry-required output declares FoundryValidationRequest.
16. Negative evidence affects later SourceFrontier ranking.
17. Full run emits EvidenceBundle and Ledger trace.

Appendix D

PlantUML Diagrams

D.1 Run automaton

```
@startuml
start
:Bind Host;
:Build HostCube;
:Compute VoidMap;
:Compute DeficiencyVector;
:Build SourceFrontierGraph;
:Acquire Sources under Capability;
:Emit SourceEvidence;
:Parse and Lower to CodeHDAG;
:Spawn SourceCubeWorkers;
:Build SourceCubes;
:Compute CubeMandorla;
if (Convergence threshold met?) then (yes)
  :Emit CompositeSupportCube;
  :Derive HostTargetDelta;
  :Extract Motifs;
  :Emit StructuralYield;
  :Run GateCascade;
  :Route CandidateEvidence;
else (no)
  :Emit Partial / Conflict / Negative Evidence;
endif
:Write Ledger and Metrics;
stop
@enduml
```

D.2 CubeSwarm

```
@startuml
HostCube --> VoidMap
VoidMap --> DeficiencyVector
```

```
DeficiencyVector --> SourceFrontier
SourceFrontier --> SourceEvidence
SourceEvidence --> SourceCubeWorker
SourceCubeWorker --> SourceCube
SourceCube --> CubeSwarm
CubeSwarm --> CubeMandorla
CubeMandorla --> MonolithicCoagulation
MonolithicCoagulation --> CompositeSupportCube
CompositeSupportCube --> HostTargetDelta
HostTargetDelta --> MotifExtraction
@enduml
```

Appendix E

Source Corpus Notes

This v0.3 monolith consolidates and refounds the following prior specification families:

- Kosmocrates Workbench Master Specification v0.1 - Wonderlamp Distillation Edition.
- Kosmocrates HYPHAE Master Specification v0.2 - Topology-Guided Corpus Assimilation Layer.
- Hypercube-HDAG Framework - Self-Compiling n-Dimensional Containers.
- Hypercube Context Decomposition - Context routing and norm compression.
- PSP / Post-Symbolic Programming specifications.
- HDAG semantic resonance model.
- MERKABA / SRI phase-resonant multi-instance orchestration model.

Closing Thesis

HYPHAE v0.3 is a topological compressor for software ecosystems. It binds one host repository as a HostCube, processes many external repositories as transient SourceCubes, synchronizes their meshes, preserves their conflicts, coagulates their shared support, extracts typed motifs, and routes only gated, evidence-bound structural candidates into the Kosmocrates Workbench.

It does not absorb code. It absorbs structure.